Generation and Sampling

Document 6 of 6 — Building a GPT from Scratch

Training teaches the model to predict the next token. Generation is where we actually use that ability: start with a prompt, predict the next token, append it, and repeat. The quality of the output depends entirely on how we select each token from the model's predicted probability distribution.

1 Autoregressive Generation

The generation procedure is a loop:

1. Start with a prompt (a sequence of tokens).
2. Feed the sequence to the model. The model outputs logits (unnormalized scores) for every position, but we only care about the logits at the last position — those are the predictions for the next token.
3. Convert logits to a probability distribution (via softmax, possibly with modifications).
4. Sample a token from that distribution.
5. Append the sampled token to the sequence.
6. Repeat from step 2.

This is inherently sequential. Each new token depends on all previous tokens (through the causal self-attention mechanism), so there is no way to generate multiple tokens in parallel. This is the fundamental speed bottleneck of autoregressive language models — during generation, you do one full forward pass per token.

When the sequence grows beyond `block_size` (256 in our model), we crop from the left:

```
idx_cond = idx if idx.size(1) <= self.config.block_size else idx[:,
    -self.config.block_size:]
```

The model can only attend to the most recent 256 tokens. Earlier tokens are simply dropped. This is a hard limitation of fixed-length positional embeddings.

Two settings matter during generation:

- **`model.eval()`** disables dropout. During training, dropout randomly zeroes activations to prevent overfitting. During generation, we want deterministic, full-capacity behavior from the model.
- **`torch.no_grad()`** disables gradient tracking. We are not computing any loss or doing any backward pass, so there is no need to store the computation graph. This saves substantial memory.

2 Greedy Decoding

The simplest strategy: always pick the token with the highest probability.

```
next_token = logits.argmax(dim=-1)
```

This is deterministic — the same input always produces the same output. Greedy decoding is what the Annotated Transformer's `greedy_decode()` function uses for machine translation, where there is typically one correct translation and you want the most likely one.

For open-ended text generation, greedy decoding produces repetitive, generic text. The model always picks the "safe" choice — the most common next character given the context. This leads to loops ("the the the...") and bland output. The model has learned a rich probability distribution over possible continuations, but greedy decoding collapses that distribution to a single point.

3 Temperature Scaling

Temperature gives us a knob to control the shape of the probability distribution before sampling.

Given logits $z_1, z_2, \ldots, z_K$ (one per vocabulary token), divide by temperature T before applying softmax:

$$P(\text{token}_i) = \frac{\exp(z_i/T)}{\sum_{j=1}^{K} \exp(z_j/T)}$$

The effect depends on T:

$T = 1.0$ (default): The distribution is unchanged. This is what the model was trained to produce.

$T \to 0$: The distribution becomes sharper. Dividing by a small T amplifies the differences between logits. If the highest logit is 5.0 and the second-highest is 4.5, dividing by $T = 0.1$ makes them 50.0 and 45.0 — the gap from 0.5 to 5.0. After softmax, the highest-logit token dominates exponentially. In the limit, this approaches greedy decoding (argmax).

$T > 1.0$: The distribution becomes flatter. Dividing by a large T compresses the differences between logits. All tokens become more equally likely. The model samples more randomly, producing more varied but less coherent text.

$T < 1.0$ (but above 0): The distribution becomes sharper than the model's default. High-probability tokens become even more dominant, low-probability tokens become even less likely. This produces more conservative, predictable text.

```
# In our generate() method:
logits = logits[:, -1, :] / temperature  # scale before softmax
probs = F.softmax(logits, dim=-1)
next_token = torch.multinomial(probs, num_samples=1)
```

Practical range: 0.5 to 1.2 covers most use cases. Below 0.5, output is nearly deterministic. Above 1.2, the model starts making frequent errors.

4 Top-k Sampling

Temperature controls the shape of the distribution, but it does not eliminate the long tail. Even at moderate temperatures, the model occasionally samples from tokens with very low probability — characters that make no sense in context. These rare samples produce garbage characters that derail the text.

Top-k sampling fixes this by restricting the sample to the k most probable tokens. All other tokens are set to $-\infty$ before softmax, which gives them zero probability.

```
if top_k is not None:
    v, _ = torch.topk(logits, min(top_k, logits.size(-1)))
    logits[logits < v[:, [-1]]] = float("-inf")

probs = F.softmax(logits, dim=-1)
next_token = torch.multinomial(probs, num_samples=1)
```

Step by step:

1. `torch.topk(logits, k)` returns the k largest logit values (and their indices).
2. `v[:, [-1]]` is the smallest value among the top k — the threshold.
3. Every logit below this threshold is set to $-\infty$.
4. Softmax converts $-\infty$ to probability 0. Only the top k tokens have nonzero probability.
5. `torch.multinomial` samples from this truncated distribution.

Typical values: $k = 40$ to $k = 200$.

The trade-off: too small a k restricts the model to only the most obvious continuations, producing repetitive text (similar to low temperature or greedy decoding). Too large a k lets through low-probability tokens that introduce noise. For a character-level model with a 65-token vocabulary, even $k = 40$ leaves most of the vocabulary available — top-k is more impactful with larger vocabularies (e.g., BPE with 50K tokens, where the tail contains many irrelevant subwords).

Temperature and top-k work well together: temperature controls the relative probabilities among the surviving tokens, while top-k eliminates the tail.

5 Top-p (Nucleus) Sampling – Concept

Top-k uses a fixed number of tokens regardless of the distribution shape. This can be suboptimal: when the model is very confident (one token has 95% probability), $k = 50$ includes 49 near-zero-probability tokens unnecessarily. When the model is uncertain (probability spread across many tokens), $k = 50$ might cut off plausible continuations.

Top-p (nucleus) sampling (Holtzman et al., 2019) adapts to the distribution. Instead of keeping a fixed count of tokens, it keeps the smallest set of tokens whose cumulative probability exceeds a threshold p:

1. Sort tokens by probability, highest first.
2. Compute cumulative probabilities: $c_1 = p_1$, $c_2 = p_1 + p_2$, etc.
3. Include tokens until $c_i \geq p$. Set all remaining tokens to $-\infty$.

For confident predictions (one token at 95%), the nucleus contains just 1–2 tokens. For uncertain predictions (many tokens at 2–5% each), the nucleus might contain 20–30 tokens. The sampling adapts automatically to the model's confidence.

Typical values: $p = 0.9$ to $p = 0.95$.

We do not implement top-p in our code — top-k is sufficient for a character-level model with a 65-token vocabulary. But top-p (often combined with temperature) is the standard method used in production language models (GPT-4, Claude, etc.).

6 The Generate Function

The complete generation pipeline is split across two files:

src/model.py contains the `generate()` method on the model class:

```
@torch.no_grad()
def generate(self, idx, max_new_tokens, temperature=1.0, top_k=None):
    for _ in range(max_new_tokens):
        # Crop to block_size if sequence too long
        idx_cond = idx if idx.size(1) <= self.config.block_size \
                       else idx[:, -self.config.block_size:]

        logits, _ = self(idx_cond)          # forward pass
        logits = logits[:, -1, :] / temperature   # last position,
            scale

        if top_k is not None:
            v, _ = torch.topk(logits, min(top_k, logits.size(-1)))
            logits[logits < v[:, [-1]]] = float("-inf")

        probs = F.softmax(logits, dim=-1)
        next_token = torch.multinomial(probs, num_samples=1)
        idx = torch.cat([idx, next_token], dim=1)

    return idx
```

src/generate.py contains the CLI script that ties everything together:

1. Load the checkpoint (model weights, config, vocabulary).
2. Reconstruct the model and tokenizer.
3. Encode the prompt string to a tensor of token IDs.
4. Call `model.generate()` to produce new tokens.
5. Decode the full sequence (prompt + generated tokens) back to a string.

```
uv run python -m src.generate --prompt "ROMEO:" --temperature 0.8
    --top-k 40
```

7 Example Outputs

The quality of generated text varies significantly with temperature. Here is what to expect from a trained BabyGPT at different settings:

Temperature 0.5 (conservative): More repetitive phrasing, better character-level spelling, tends to stick to common patterns. The model produces text that looks structurally correct but lacks variety. Dialogue formatting (character names followed by colons) is usually correct. The same phrases may repeat.

Temperature 0.8 (recommended): A good balance between coherence and variety. The model produces varied text with mostly correct spelling and formatting. Individual lines look plausible as Shakespeare-like dialogue. This is the default in our `src/generate.py`.

Temperature 1.2 (creative): More surprising word choices and character combinations, but more spelling errors and occasional nonsense characters. The text is more varied but less consistently well-formed. Occasional stretches of gibberish.

Important: this is a **character-level** model trained on 1MB of text. The model learns:

- Character-level patterns: common letter sequences, word boundaries, capitalization
- Formatting conventions: character names in uppercase followed by colons, verse-like line breaks, act/scene headers
- Pseudo-Elizabethan spelling: sequences like "thou", "hath", "thy" appear frequently because they are statistically common

The model does **not** learn:

- Semantic coherence: sentences do not form a logical narrative
- Plot or character consistency: the model does not track who is speaking or what they are talking about
- Factual content: the model cannot reference actual Shakespeare plots

This is not a limitation of the architecture — it is a limitation of scale. The same architecture, with more parameters, subword tokenization, and more data, produces GPT-3.

8 What Would Make It Better

Our BabyGPT is a minimal working example. Each of the following changes would produce measurable improvements, and they are roughly ordered by impact:

More data. TinyShakespeare is 1MB. GPT-2 was trained on 40GB (WebText). GPT-3 on 570GB (filtered Common Crawl + books + Wikipedia). More data means more patterns to learn and less overfitting. This is the single most impactful change.

Subword tokenization (BPE). Character-level tokenization means the model must use its context window to learn that "t", "h", "e" form the word "the." With BPE, "the" is a single token, and the model can use its context window for higher-level patterns. A 256-token context window covers roughly 1000 characters with BPE versus 256 characters with character-level tokenization.

Larger model. More layers and more embedding dimensions increase the model's capacity to represent patterns. Scaling laws (Kaplan et al., 2020) show that loss decreases as a power law with model size, training data, and compute — and these three factors are roughly interchangeable. For a fixed compute budget, there is an optimal balance between model size and data size (Hoffmann et al., 2022 — the Chinchilla scaling laws).

More compute (longer training). 5000 iterations over 1M tokens is fast but leaves room for further optimization. Longer training with more data would continue to reduce loss.

These are exactly the insights behind the GPT scaling papers. The architecture we built is essentially the same architecture (up to minor details like attention implementations and normalization choices) used in models with 100 billion+ parameters. The difference is entirely in scale: more parameters, more data, more compute.

This concludes the six-document series. The full pipeline — from the attention mechanism to a trained, generating language model — is implemented in `src/` and explained in `learn/`.